## 一．文本聚类

文本聚类的步骤：1. 文本嵌入，获取输入文档进行转换，将输入文档转换为向量。

```
!pip install bertopic datasets openai datamapplot
```

```
!pip install --upgrade datasets
```

```
#下载数据集
from datasets import load_dataset
dataset = load_dataset("maartengr/arxiv_nlp")["train"]

#提取原数据
abstracts = dataset["Abstracts"]
titles = dataset["Titles"]
```

```
/usr/local/lib/python3.11/dist-packages/huggingface_hub/utils/_auth.py:94: UserWarning:
The secret `HF_TOKEN` does not exist in your Colab secrets.
To authenticate with the Hugging Face Hub, create a token in your settings tab (https://huggin
You will be able to reuse this secret in all of your notebooks.
Please note that authentication is recommended but still optional to access public models or d
  warnings.warn(
README.md: 100% 617/617 [00:00<00:00, 23.8kB/s]
data.csv: 100% 53.2M/53.2M [00:00<00:00, 119MB/s]
Generating train split: 44949/0 00:00<00:00, 70946.77 examples/s]
```

```python
#调用模型
from sentence_transformers import SentenceTransformer
#从 sentence_transformers库中导入 SentenceTransformer类，这是一个专门用于生成句子、段落或文本嵌入的库

#Create an embedding for each abstract

embedding_model = SentenceTransformer('thenlper/gte-small')
#加载预训练模型，是一个开源文本嵌入模型，适用于通用语义检索任务，模型会将输入文本映射到一个高维向量空间（384维）

embeddings = embedding_model.encode(abstracts,show_progress_bar=True)
#对每个文本调用模型的 encode()方法，将其转换为向量；show_progress_bar=True 会显示处理进度条
```

```
/usr/local/lib/python3.11/dist-packages/huggingface_hub/utils/_auth.py:94: UserWarning:
The secret `HF_TOKEN` does not exist in your Colab secrets.
To authenticate with the Hugging Face Hub, create a token in your settings tab (https://huggingface.co/settings/token
You will be able to reuse this secret in all of your notebooks.
Please note that authentication is recommended but still optional to access public models or datasets.
  warnings.warn(
modules.json: 100% 385/385 [00:00<00:00, 35.6kB/s]
README.md: 100% 68.1k/68.1k [00:00<00:00, 2.32MB/s]
sentence_bert_config.json: 100% 57.0/57.0 [00:00<00:00, 4.51kB/s]
config.json: 100% 583/583 [00:00<00:00, 53.5kB/s]
model.safetensors: 100% 66.7M/66.7M [00:00<00:00, 147MB/s]
tokenizer_config.json: 100% 394/394 [00:00<00:00, 41.4kB/s]
vocab.txt: 100% 232k/232k [00:00<00:00, 7.58MB/s]
tokenizer.json: 100% 712k/712k [00:00<00:00, 5.03MB/s]
special_tokens_map.json: 100% 125/125 [00:00<00:00, 10.1kB/s]
config.json: 100% 190/190 [00:00<00:00, 20.5kB/s]
```

```
Batches: 100% 1405/1405 [03:44<00:00, 18.84it/s]
```

2. 减小向量的维度；

```python
#减小向量的维度
from umap import UMAP

#将输入嵌入从384维降低到5维
umap_model = UMAP(
        n_components=5,min_dist=0.0,metric='cosine',random_state=42
)
reduced_embeddings = umap_model.fit_transform(embeddings)
```

3. 对降维后的嵌入进行聚类

使用 HDBSCAN 算法 对降维后的文本嵌入向量进行聚类分析，目的是将语义相似的文本分组。HDBSCAN 是一种基于密度的层次聚类算法，擅长处理噪声和发现任意形状的簇。

```python
#将降维后的嵌入进行聚类
from hdbscan import HDBSCAN

#我们拟合模型并提取聚类
hdbscan_model = HDBSCAN(
        min_cluster_size=50,  #定义簇的最小大小（核心参数），若一个簇的样本数 〈50，会被视为噪声或合并到其他簇
        metric='euclidean',  #使用欧氏距离衡量向量间的相似性
        cluster_selection_method='eom'  #簇选择方法为eom
).fit(reduced_embeddings)  #输入降维后的嵌入向量

clusters = hdbscan_model.labels_  #返回每个样本的簇标签，格式为整数数组

#打印聚类结果，簇的数量
len(set(clusters))
```

```
/usr/local/lib/python3.11/dist-packages/sklearn/utils/deprecation.py:151: FutureWarning: 'force_all_finite' wa
  warnings.warn(
/usr/local/lib/python3.11/dist-packages/sklearn/utils/deprecation.py:151: FutureWarning: 'force_all_finite' wa
  warnings.warn(
155
```

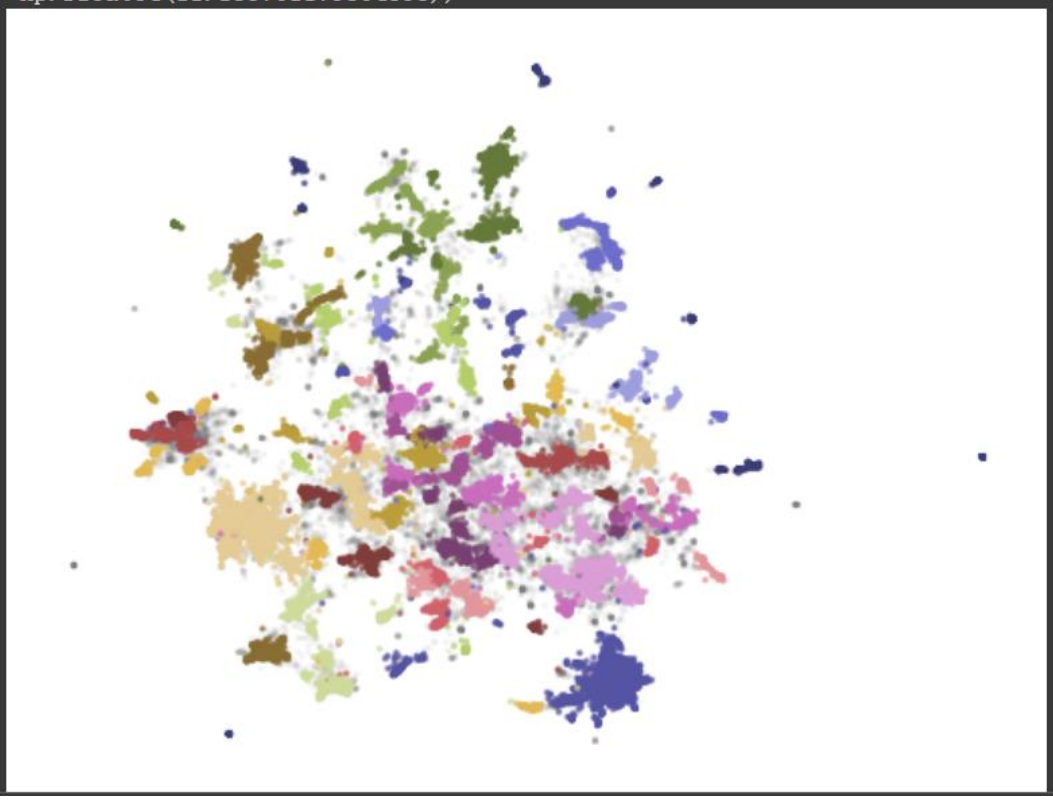
```python
import numpy as np

#打印第一个簇中的前三个文本
cluster = 0
for index in np.where(clusters==cluster)[0][:3]:
    print(abstracts[index][:300] +"...\n")
```

```
  This works aims to design a statistical machine translation from English text
to American Sign Language (ASL). The system is based on Moses tool with some
modifications and the results are synthesized through a 3D avatar for
interpretation. First, we translate the input text to gloss, a written fo...

  Researches on signed languages still strongly dissociate lin- guistic issues
related on phonological and phonetic aspects, and gesture studies for
recognition and synthesis purposes. This paper focuses on the imbrication of
motion and meaning for the analysis, synthesis and evaluation of sign lang...

  Modern computational linguistic software cannot produce important aspects of
sign language translation. Using some researches we deduce that the majority of
automatic sign language translation systems ignore many aspects when they
generate animation; therefore the interpretation lost the truth inf...
```

```
#可视化成二维或三维
import pandas as pd

reduced_embeddings = UMAP(
        n_components=2,  #指定维数2或3
        min_dist=0.0,  #控制点之间的最小间距，0表示允许点重叠
        metric='cosine',
        random_state=42  #固定随机种子
).fit_transform(embeddings)  #二维数组，size为（44949,2）

#Create dataframe
#将降维后的坐标、标题和聚类标签整合到一个DataFrame 中，便于后续分析和可视化
df = pd.DataFrame(reduced_embeddings,columns=["x","y"])
df["title"] = titles
df["cluster"] = [str(c) for c in clusters]

#选择异常值与非异常值（簇）
clusters_df = df.loc[df.cluster !="-1",:] #-1为异常值样本的label，非-1为有效簇的样本label
outliers_df = df.loc[df.cluster =="-1",:]
```

```
import  matplotlib.pyplot  as  plt

#分别绘制异常值和非异常值
plt.scatter(outliers_df.x,  outliers_df.y,  alpha=0.05,  s=2,  c="grey")
#将聚类中被标记为噪声（cluster=-1）的样本以浅灰色小点显示

plt.scatter(
        clusters_df.x,  clusters_df.y,
        c=clusters_df.cluster.astype(int),#将簇标签转换为整数，作为颜色索引
        alpha=0.6,  s=2,
        cmap='tab20b'  #指定颜色映射，支持多达20种颜色
)
plt.axis('off')
```

```
(np.float64(-8.334335255622864),
 np.float64(11.817492890357972),
 np.float64(-5.973596382141113),
 np.float64(11.185701179504395))
```



## 二．主题建模

目的：提取它们的标题进行聚类，并给每个聚类分配了名称。

步骤：1.用 CountVectorizer 计算（c-TF-IDF 算法）每个簇中的词频 TF，然后将词频与逆文档频率（逆文档频率 IDF）进行相乘，计算每个簇的词频-逆文档频

率矩阵 TF-IDF.

TF (Term Frequency)： 衡量一个词在**单篇文档**中的重要性（出现次数越多，在该文档中可能越重要）。

IDF(Inverse Document Frequency)：衡量一个词在整个文档集合中（所有簇）中出现的频率。IDF 是 DF 的**倒数**（取对数），所以某个词的 DF 越高，IDF 越低，代表这个词在非常多文档中出现，所以对这个词的关注少，分配到的权重小。反之，某个词的 DF 越低，IDF 越高，代表这个词在很少文档中出现，它能够代表某个簇，可以成为这些簇的关键词，所以对这个词的关注多，分配到的权重大。

TF-IDF 给那些在**当前文档中出现频繁（TF 高）** 且在整个文档集合中**相对罕见（IDF 高）** 的词赋予很高的权重。这些词最能代表该文档的**独特内容**。也就是说，对于关键词，它的 TF 越高越好，同时 IDF 越高越好。

1. 下载数据集并提取 titles 和 abstracts

```
!pip install bertopic datasets openai datamapplot
```

```
!pip install datasets --upgrade
```

```
#从hugging face下载数据集
from datasets import load_dataset
dataset = load_dataset("maartengr/arxiv_nlp")["train"]

#提取元数据
abstracts = dataset["Abstracts"]
titles = dataset["Titles"]
```

```
/usr/local/lib/python3.11/dist-packages/huggingface_hub/utils/_auth.py:94: UserWarning:
The secret `HF_TOKEN` does not exist in your Colab secrets.
To authenticate with the Hugging Face Hub, create a token in your settings tab (https://huggingface.co/
You will be able to reuse this secret in all of your notebooks.
Please note that authentication is recommended but still optional to access public models or datasets.
  warnings.warn(
README.md: 100% 617/617 [00:00<00:00, 13.4kB/s]
data.csv: 100% 53.2M/53.2M [00:00<00:00, 161MB/s]
Generating train split: 44949/0 [00:01<00:00, 28387.27 examples/s]
```

2. 将输入文档转换为高维向量

```
from sentence_transformers import SentenceTransformer

#为每一个摘要创造嵌入
embedding_model = SentenceTransformer("thenlper/gte-small")
embeddings = embedding_model.encode(abstracts, show_progress_bar=True)
```

```
Batches: 100% 1405/1405 [03:33<00:00, 22.44it/s]
```

3. 将高维向量降维处理

```
from umap import UMAP

#将输入嵌入从384维降低到5维
umap_model = UMAP(
        n_components=5, min_dist=0.0, metric='cosine', random_state=42
)

reduced_embeddings = umap_model.fit_transform(embeddings)
```

4. 对降维后的向量聚类

```
from hdbscan import HDBSCAN

#拟合模型并提取聚类
hdbscan_model = HDBSCAN(
        min_cluster_size=50,
        metric='euclidean',
        cluster_selection_method='eom'
).fit(reduced_embeddings)

clusters = hdbscan_model.labels_  #返回每个样本的簇标签，格式为整数数组
```

5. 训练主题建模模型

所有组件（embedding_model /umap_model /hdbscan_model）需预先配置（上述代码所示）。

```
from bertopic import BERTopic

#用之前定义的模型训练新模型：使用 BERTopic 库训练一个主题建模模型
topic_model = BERTopic(
        embedding_model=embedding_model,
        umap_model=umap_model,  #降维模型，可以用PCA替换UMAP
        hdbscan_model=hdbscan_model,  #聚类模型，可以用k-means替换hdbscan
        verbose=True
).fit(abstracts, embeddings)  #输入摘要和高维向量
```

6. 查看主题建模内容

```
topic_model.get_topic_info()
```

| | Topic | Count | Name | Representation | Representative_Docs |
|---|---|---|---|---|---|
| 0 | -1 | 14210 | -1_of_the_and_to | [of, the, and, to, in, we, language, for, that... | [ GPT-$3$ has attracted lots of attention due... |
| 1 | 0 | 2316 | 0_speech_asr_recognition_end | [speech, asr, recognition, end, acoustic, spea... | [ The speech chain mechanism integrates autom... |
| 2 | 1 | 2183 | 1_question_qa_questions_answer | [question, qa, questions, answer, answering, a... | [ Text-based Question Generation (QG) aims at... |
| 3 | 2 | 941 | 2_translation_nmt_machine_bleu | [translation, nmt, machine, bleu, neural, engl... | [ Recently, the development of neural machine... |
| 4 | 3 | 880 | 3_summarization_summaries_summary_abstractive | [summarization, summaries, summary, abstractiv... | [ Sentence summarization shortens given texts... |
| ... | ... | ... | ... | ... | ... |
| 150 | 149 | 54 | 149_sentence_embeddings_sts_embedding | [sentence, embeddings, sts, embedding, similar... | [ Sentence representation at the semantic lev... |
| 151 | 150 | 54 | 150_gans_gan_adversarial_generation | [gans, gan, adversarial, generation, generativ... | [ Text generation is of particular interest i... |
| 152 | 151 | 54 | 151_coherence_discourse_paragraph_text | [coherence, discourse, paragraph, text, cohesi... | [ While there has been significant progress t... |
| 153 | 152 | 53 | 152_chatgpt_its_openai_tasks | [chatgpt, its, openai, tasks, has, ai, capabil... | [ Over the last few years, large language mod... |
| 154 | 153 | 52 | 153_opinion_reviews_summaries_summarization | [opinion, reviews, summaries, summarization, r... | [ When faced with a large number of product r... |

155 rows × 5 columns

可以根据这个表格查看每个簇并打印出相关的关键词

```
#打印与第11个簇相关的关键词
topic_model.get_topic(11)
```

```
[('parsing', np.float64(0.04075430955451941)),
 ('dependency', np.float64(0.0335918911692885)),
 ('parser', np.float64(0.026405190580623305)),
 ('parsers', np.float64(0.01906459384296242)),
 ('treebank', np.float64(0.015824337720041225)),
 ('trees', np.float64(0.015126010102075307)),
 ('transition', np.float64(0.014436295818998192)),
 ('treebanks', np.float64(0.012776168311711 86)),
 ('constituency', np.float64(0.012466041865879487)),
 ('syntactic', np.float64(0.011417735960230217))]
```

还可以用 find_topics()函数,用特定的搜索词去查找指定的 topics。

```
topic_model.find_topics("topic  modeling")
```

```
([24, -1, 38, 32, 84],
 [np.float32(0.9545274),
  np.float32(0.9123676),
  np.float32(0.9080541),
  np.float32(0.9053283),
  np.float32(0.90453553)])
```

簇-1，24,32,38 和 84 都与主题建模“topic modeling”相关，簇-1 是噪声的集

合，簇 24 的相似度最高，为 0.9545.接下来我们可以查看簇 24 的内容。

```
topic_model.get_topic(24)
```

```
[('topic', np.float64(0.06811153301756999)),
 ('topics', np.float64(0.035746717561104396)),
 ('lda', np.float64(0.016020062969070364)),
 ('latent', np.float64(0.013574936227317968)),
 ('documents', np.float64(0.013201698266173009)),
 ('document', np.float64(0.012912590658853182)),
 ('modeling', np.float64(0.012084716289729468)),
 ('dirichlet', np.float64(0.01010281253111858)),
 ('word', np.float64(0.008653858081603273)),
 ('allocation', np.float64(0.007950503995528465))]
```

如果我们知道某个特定主题的论文标题，我们可以查找它是否在某个特定主题的簇中。

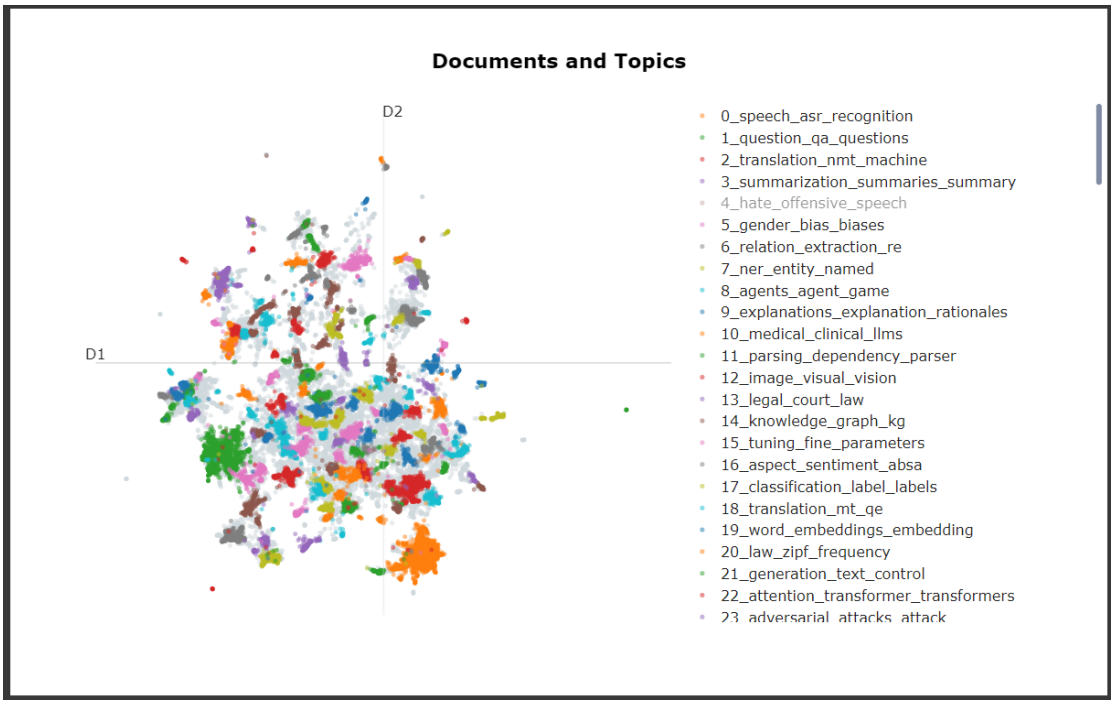
```
topic_model.topics_[titles.index('BERTopic: Neural topic modeling with a class-based TF-IDF procedure')]
```

```
24
```

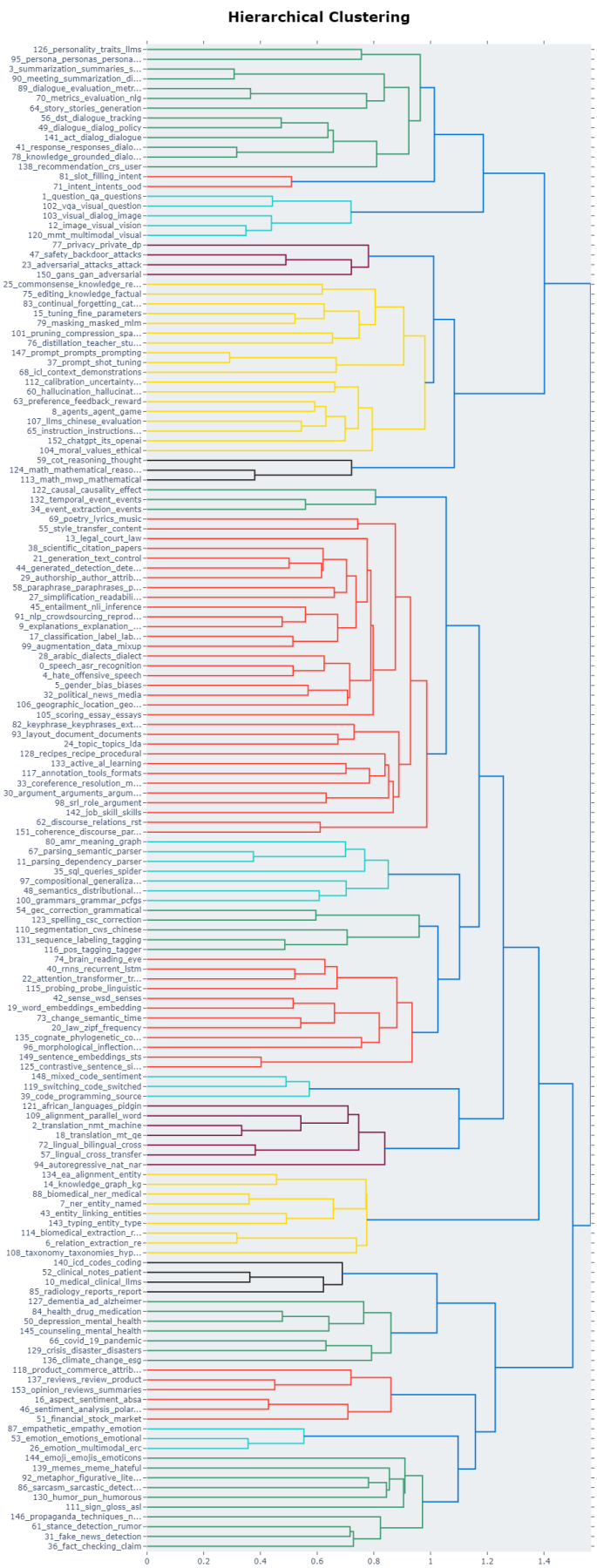
```
topic_model.topics_[titles.index('Attention Is All You Need')]
```

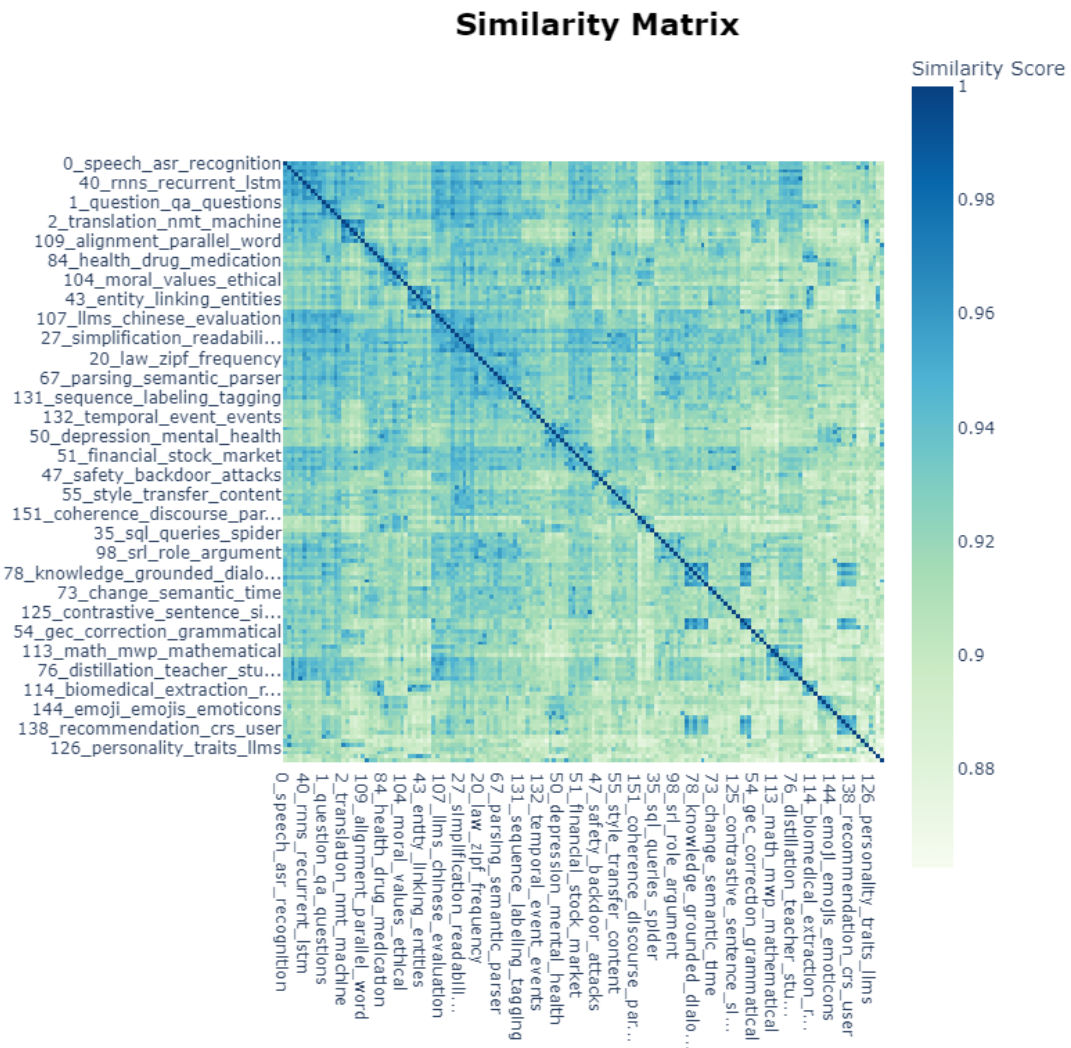
```
2
```

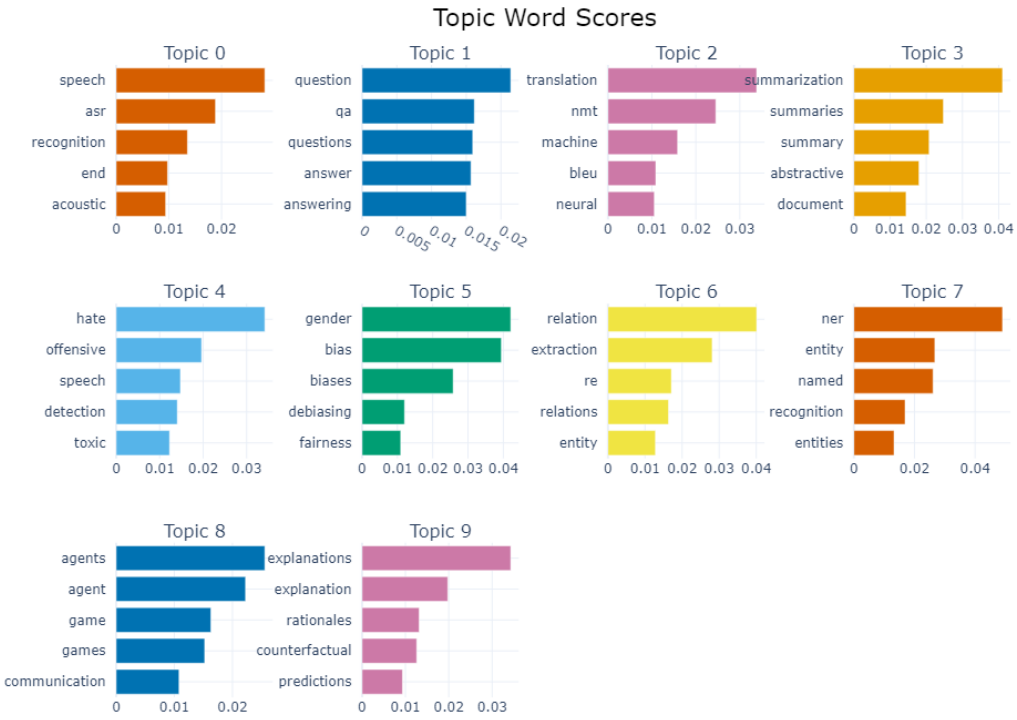
```
reduced_embeddings = UMAP(
        n_components=2,  #指定维数2或3
        min_dist=0.0,  #控制点之间的最小间距，0表示允许点重叠
        metric='cosine',
        random_state=42  #固定随机种子
).fit_transform(embeddings)  #二维数组，size为（44949,2）

#将主题和文档可视化
#visualize_documents创建交互式2D散点图，每个点代表一个文档，
#点的颜色表示文档所属主题，鼠标悬停显示文档标题（titles）
fig = topic_model.visualize_documents(
        titles,
        reduced_embeddings=reduced_embeddings,
        width=1200,
        hide_annotations=True
)

#更新图例字体，更容易可视化
fig.update_layout(font=dict(size=16))
```




```
#可视化条形图与排名关键字
topic_model.visualize_barchart(top_n_topics=10)

#主题之间的可视化关系
topic_model.visualize_heatmap(n_clusters=30)

#可视化主题的潜在层次结构
topic_model.visualize_hierarchy()
```


Hierarchical Clustering
126_personality_traits_llms
95_persona_personas_persona...
3_summarization_summaries_s...
90_meeting_summarization_di...
89_dialogue_evaluation_metr...
70_metrics_evaluation_nlg
64_story_stories_generation
56_dst_dialogue_tracking
49_dialogue_dialog_policy
141_act_dialog_dialogue
41_response_responses_dialo...
78_knowledge_grounded_dialo...
138_recommendation_crs_user
81_slot_filling_intent
71_intent_intents_ood
1_question_qa_questions
102_vqa_visual_question
103_visual_dialog_image
12_image_visual_vision
120_mmt_multimodal_visual
77_privacy_private_dp
47_safety_backdoor_attacks
23_adversarial_attacks_attack
150_gans_gan_adversarial
25_commonsense_knowledge_re...
75_editing_knowledge_factual
83_continual_forgetting_cat...
15_tuning_fine_parameters
79_masking_masked_mlm
101_pruning_compression_spa...
76_distillation_teacher_stu...
147_prompt_prompts_prompting
37_prompt_shot_tuning
68_icl_context_demonstrations
112_calibration_uncertainty...
60_hallucination_hallucinat...
63_preference_feedback_reward
8_agents_agent_game
107_llms_chinese_evaluation
65_instruction_instructions...
152_chatgpt_its_openai
104_moral_values_ethical
59_cot_reasoning_thought
124_math_mathematical_reaso...
113_math_mwp_mathematical
122_causal_causality_effect
132_temporal_event_events
34_event_extraction_events
69_poetry_lyrics_music
55_style_transfer_content
13_legal_court_law
38_scientific_citation_papers
21_generation_text_control
44_generated_detection_dete...
29_authorship_author_attrib...
58_paraphrase_paraphrases_p...
27_simplification_readabili...
45_entailment_nli_inference
91_nlp_crowdsourcing_reprod...
9_explanations_explanation_...
17_classification_label_lab...
99_augmentation_data_mixup
28_arabic_dialects_dialect
0_speech_asr_recognition
4_hate_offensive_speech
5_gender_bias_biases
32_political_news_media
106_geographic_location_geo...
105_scoring_essay_essays
82_keyphrase_keyphrases_ext...
93_layout_document_documents
24_topic_topics_lda
128_recipes_recipe_procedural
133_active_al_learning
117_annotation_tools_formats
33_coreference_resolution_m...
30_argument_arguments_argum...
98_srl_role_argument
142_job_skill_skills
62_discourse_relations_rst
151_coherence_discourse_par...
80_amr_meaning_graph
67_parsing_semantic_parser
11_parsing_dependency_parser
35_sql_queries_spider
97_compositional_generaliza...
48_semantics_distributional...
100_grammars_grammar_pcfgs
54_gec_correction_grammatical
123_spelling_csc_correction
110_segmentation_cws_chinese
131_sequence_labeling_tagging
116_pos_tagging_tagger
74_brain_reading_eye
40_rnns_recurrent_lstm
22_attention_transformer_tr...
115_probing_probe_linguistic
42_sense_wsd_senses
19_word_embeddings_embedding
73_change_semantic_time
20_law_zipf_frequency
135_cognate_phylogenetic_co...
96_morphological_inflection...
149_sentence_embeddings_sts
125_contrastive_sentence_si...
148_mixed_code_sentiment
119_switching_code_switched
39_code_programming_source
121_african_languages_pidgin
109_alignment_parallel_word
2_translation_nmt_machine
18_translation_mt_qe
72_lingual_bilingual_cross
57_lingual_cross_transfer
94_autoregressive_nat_nar
134_ea_alignment_entity
14_knowledge_graph_kg
88_biomedical_ner_medical
7_ner_entity_named
43_entity_linking_entities
143_typing_entity_type
114_biomedical_extraction_r...
6_relation_extraction_re
108_taxonomy_taxonomies_hyp...
140_icd_codes_coding
52_clinical_notes_patient
10_medical_clinical_llms
85_radiology_reports_report
127_dementia_ad_alzheimer
84_health_drug_medication
50_depression_mental_health
145_counseling_mental_health
66_covid_19_pandemic
129_crisis_disaster_disasters
136_climate_change_esg
118_product_commerce_attrib...
137_reviews_review_product
153_opinion_reviews_summaries
16_aspect_sentiment_absa
46_sentiment_analysis_polar...
51_financial_stock_market
87_empathetic_empathy_emotion
53_emotion_emotions_emotional
26_emotion_multimodal_erc
144_emoji_emojis_emoticons
139_memes_meme_hateful
92_metaphor_figurative_lite...
86_sarcasm_sarcastic_detect...
130_humor_pun_humorous
111_sign_gloss_asl
146_propaganda_techniques_n...
61_stance_detection_rumor
31_fake_news_detection
36_fact_checking_claim
0
0.2
0.4
0.6
0.8
1
1.2
1.4


Similarity Matrix
Similarity Score
1
0.98
0.96
0.94
0.92
0.9
0.88
0_speech_asr_recognition
40_rnns_recurrent_lstm
1_question_qa_questions
2_translation_nmt_machine
109_alignment_parallel_word
84_health_drug_medication
104_moral_values_ethical
43_entity_linking_entities
107_llms_chinese_evaluation
27_simplification_readabili...
20_law_zipf_frequency
67_parsing_semantic_parser
131_sequence_labeling_tagging
132_temporal_event_events
50_depression_mental_health
51_financial_stock_market
47_safety_backdoor_attacks
55_style_transfer_content
151_coherence_discourse_par...
35_sql_queries_spider
98_srl_role_argument
78_knowledge_grounded_dialo...
73_change_semantic_time
125_contrastive_sentence_si...
54_gec_correction_grammatical
113_math_mwp_mathematical
76_distillation_teacher_stu...
114_biomedical_extraction_r...
144_emoji_emojis_emoticons
138_recommendation_crs_user
126_personality_traits_llms


Topic Word Scores
Topic 0
speech
asr
recognition
end
acoustic
Topic 1
question
qa
questions
answer
answering
Topic 2
translation
nmt
machine
bleu
neural
Topic 3
summarization
summaries
summary
abstractive
document
Topic 4
hate
offensive
speech
detection
toxic
Topic 5
gender
bias
biases
debiasing
fairness
Topic 6
relation
extraction
re
relations
entity
Topic 7
ner
entity
named
recognition
entities
Topic 8
agents
agent
game
games
communication
Topic 9
explanations
explanation
rationales
counterfactual
predictions

现有模型的表示依然采用词袋模型，未能考虑语义结构。在聚类模型和主题建模模型上整合表示模型和生成模型，实现在处理主题时考虑语意结构。

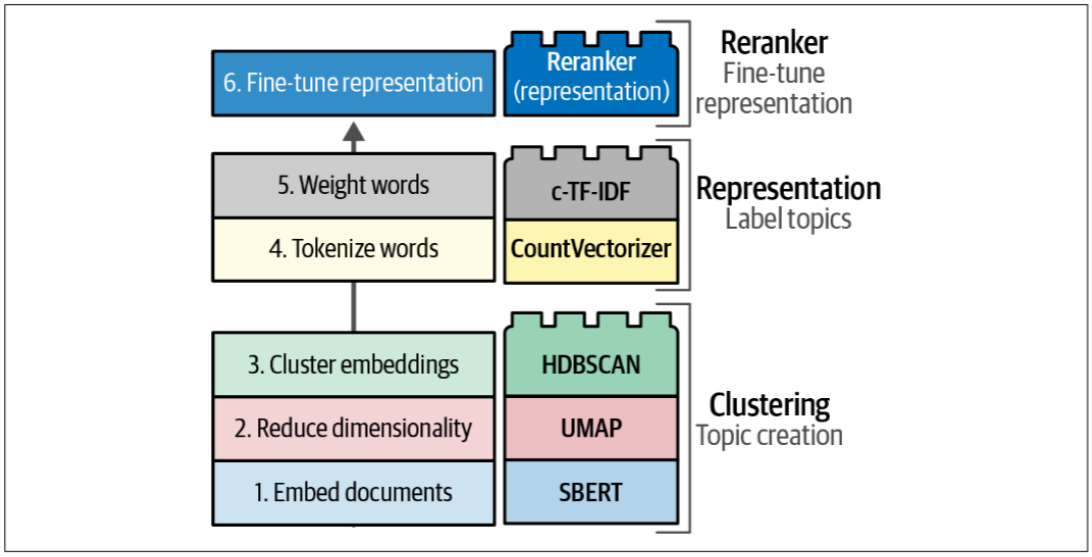



*Figure 5-20. The reranker (representation) block sits on top of the c-TF-IDF representation.*

Reranker：重新排序关键词，使关键词能够反应出它们的含义，更接近聚类中文档的含义。

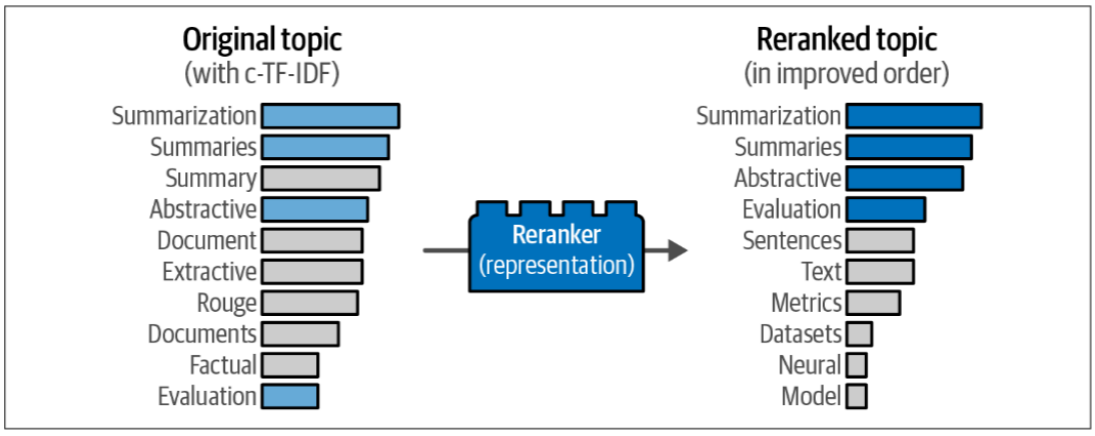



*Figure 5-19. Fine-tune the topic representations by reranking the original c-TF-IDF distributions.*

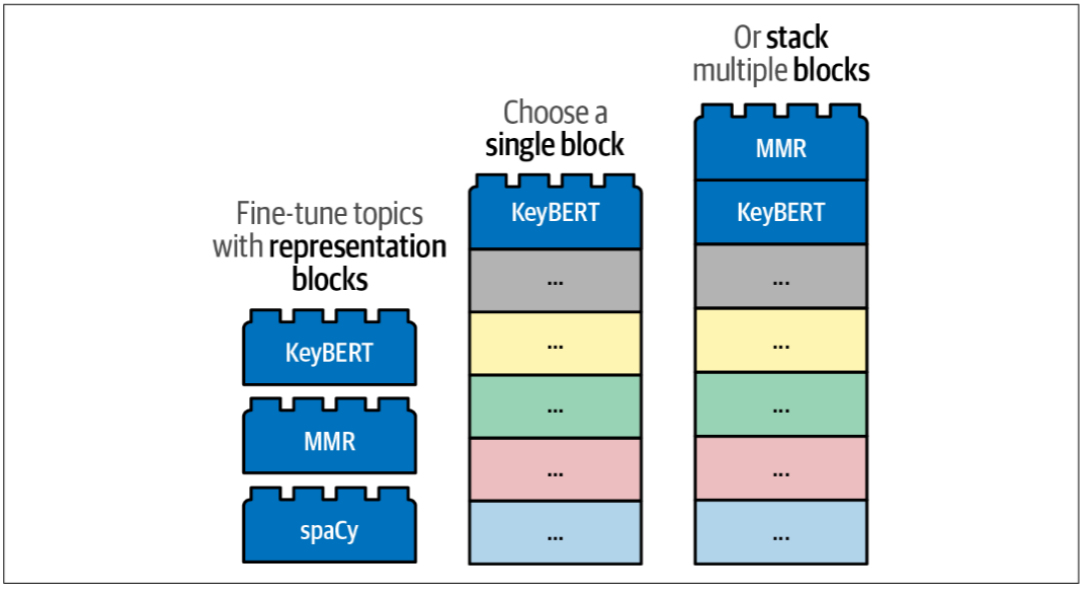



*Figure 5-21. After applying the c-TF-IDF weighting, topics can be fine-tuned with a wide variety of representation models, many of which are large language models.*

1. 使用表示模型对关键词进行重排序：可以堆叠 KeyBERT 模块和 MMR 模块。KeyBERT 模块的实现工具为 KeyBERT Inspired，在已经获得聚类关键词的基础上，对该簇中的所有文档生成一个平均嵌入向量，通过比较候选关键词和平均嵌入向量来实现关键词的重排序。

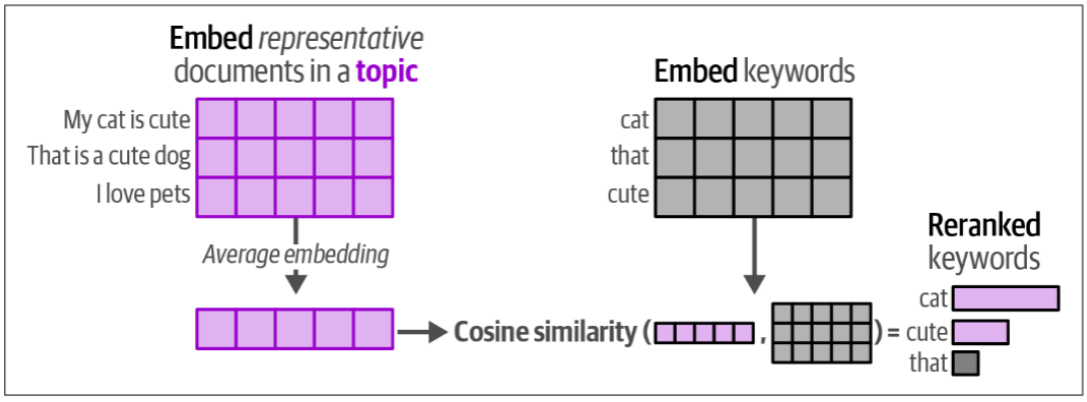



*Figure 5-22. KeyBERTInspired representation model procedure.*

在使用 c-TF-IDF 和前述 KeyBERT 启发的技术时，生成的主题表征仍存在显著冗余。我们可以采用最大边际相关性（MMR）算法来提升主题表征的多样性。该算法通过以下方式运作：首先嵌入一组候选关键词，然后迭代计算下一个最优添加关键词，从而寻找既彼此差异显著又与对比文档保持关联的关键词集合。实施过程中需要设定多样性参数以控制关键词间的差异程度。在 BERTopic 框架中，MMR 被用于将初始关键词集合（例如 30 个）精简为规模更小但多样性更强的子集（例

如 10 个），其核心机制是过滤冗余词汇，仅保留能为主题表征提供新增信息的词汇。

原始结果：

```
topic_model.get_topic_info()
```

| | Topic | Count | Name | Representation | Representative_Docs |
|---|---|---|---|---|---|
| 0 | -1 | 14830 | -1_the_of_and_to | [the, of, and, to, in, we, for, language, that... | [ Word Representations form the core componen... |
| 1 | 0 | 2203 | 0_question_qa_answer_questions | [question, qa, answer, questions, answering, a... | [ With the development of deep learning techn... |
| 2 | 1 | 1972 | 1_speech_asr_recognition_end | [speech, asr, recognition, end, acoustic, spea... | [ End-to-end Speech Translation (ST) models h... |
| 3 | 2 | 859 | 2_hate_offensive_speech_detection | [hate, offensive, speech, detection, toxic, so... | [ With growing role of social media in shapin... |
| 4 | 3 | 848 | 3_summarization_summaries_summary_abstractive | [summarization, summaries, summary, abstractiv... | [ Pre-trained neural abstractive summarizatio... |
| ... | ... | ... | ... | ... | ... |
| 150 | 149 | 53 | 149_counseling_mental_therapy_health | [counseling, mental, therapy, health, psychoth... | [ Mental health care poses an increasingly se... |
| 151 | 150 | 53 | 150_mixed_code_sentiment_mixing | [mixed, code, sentiment, mixing, english, anal... | [ The usage of more than one language in the ... |
| 152 | 151 | 53 | 151_prompt_prompts_optimization_prompting | [prompt, prompts, optimization, prompting, llm... | [ Prompt optimization aims to find the best p... |
| 153 | 152 | 50 | 152_long_context_window_length | [long, context, window, length, llms, contexts... | [ Extending the context window of large langu... |
| 154 | 153 | 50 | 153_chatgpt_its_openai_has | [chatgpt, its, openai, has, ai, tasks, respons... | [ Recently, ChatGPT has attracted great atten... |

155 rows × 5 columns

（1） KeyBERTInspired

```python
#保存原始模型（没有使用生成模型），以便后续对比分析
from copy import deepcopy
original_topics = deepcopy(topic_model.topic_representations_)
```

```python
#可视化主题词差异
def topic_differences(model, original_topics, nr_topics=5):
    """Show the differences in topic representations between two models"""
    df = pd.DataFrame(columns=["Topic", "Original", "Updated"])#创建结果表格
    for topic in range(nr_topics):

        #提取两个模型中每个簇的前五个词
        og_words = "|".join(list(zip(*original_topics[topic]))[0][:5])
        new_words = "|".join(list(zip(*model.get_topic(topic)))[0][:5])
        df.loc[len(df)] = [topic, og_words, new_words] #将结果添加到表格

    return df
```

```python
import pandas as pd
from bertopic.representation import KeyBERTInspired

#使用KeyBERTInspired更新主题表示，优化BERTopic的表示
representation_model = KeyBERTInspired()
#topic_model是之前用BERTopic库训练的主题建模模型
    #topic_model.update_topics用来更新模型
topic_model.update_topics(abstracts, representation_model=representation_model)

#展示主题差异
topic_differences(topic_model, original_topics)
```

| | Topic | Original | Updated |
|---|---|---|---|
| 0 | 0 | question\|qa\|answer\|questions\|answering | questions\|answering\|comprehension\|question\|answer |
| 1 | 1 | speech\|asr\|recognition\|end\|acoustic | translation\|speech\|transcription\|phonetic\|voice |
| 2 | 2 | hate\|offensive\|speech\|detection\|toxic | hate\|hateful\|language\|classifiers\|twitter |
| 3 | 3 | summarization\|summaries\|summary\|abstractive\|do... | summarization\|summarizers\|summaries\|summary\|se... |
| 4 | 4 | gender\|bias\|biases\|debiasing\|fairness | gender\|gendered\|bias\|biases\|biased |

（2）MMR 最大边际相关

```
from bertopic.representation import MaximalMarginalRelevance

# Update our topic representations to MaximalMarginalRelevance
representation_model = MaximalMarginalRelevance(diversity=0.5)
topic_model.update_topics(abstracts, representation_model=representation_model)

# Show topic differences
topic_differences(topic_model, original_topics)
```

| | Topic | Original | Updated |
|---|---|---|---|
| 0 | 0 | question\|qa\|answer\|questions\|answering | questions\|retrieval\|comprehension\|knowledge\|to |
| 1 | 1 | speech\|asr\|recognition\|end\|acoustic | speech\|asr\|model\|automatic\|training |
| 2 | 2 | hate\|offensive\|speech\|detection\|toxic | hate\|toxic\|social\|comments\|platforms |
| 3 | 3 | summarization\|summaries\|summary\|abstractive\|do... | summarization\|extractive\|factual\|sentences\|eva... |
| 4 | 4 | gender\|bias\|biases\|debiasing\|fairness | gender\|bias\|fairness\|stereotypes\|embeddings |

2. 使用**生成模型**为主题生成标签：模型基于先前生成的关键字和一小组代表性文档生成一个短标签，而不是生成或重新排序关键字。

使用文本生成模型(Flan-T5)重新解释主题模型中的每个主题，生成更自然、更易理解的主题描述。

```
from transformers import pipeline
from bertopic.representation import TextGeneration

prompt = """I have a topic that contains the following documents:
[DOCUMENTS]

The topic is described by the following keywords: '[KEYWORDS]'.

Based on the documents and keywords, what is this topic about?"""

# Update our topic representations using Flan-T5
generator = pipeline("text2text-generation", model="google/flan-t5-small")
representation_model = TextGeneration(
        generator, prompt=prompt, doc_length=50, tokenizer="whitespace"
)
topic_model.update_topics(abstracts, representation_model=representation_model)

# Show topic differences
topic_differences(topic_model, original_topics)
```

对每个主题：

1. 选取代表性文档
2. 提取主题关键词
3. 填充提示模板

4. 发送到 Flan-T5 模型
5. 解析返回的标签格式 topic: 生成标签
6. 用生成标签更新主题表示

| | Topic | Original | Updated |
|---|---|---|---|
| 0 | 0 | question\|qa\|answer\|questions\|answering | Question answering is one of the most importan... |
| 1 | 1 | speech\|asr\|recognition\|end\|acoustic | Speech-to-speech comparison metric\|\|\|\| |
| 2 | 2 | hate\|offensive\|speech\|detection\|toxic | Science/Tech\|\|\|\| |
| 3 | 3 | summarization\|summaries\|summary\|abstractive\|do... | Document summarization\|\|\|\| |
| 4 | 4 | gender\|bias\|biases\|debiasing\|fairness | Gender bias in artificial intelligence and nat... |

接入 deepseek API 生成

```
import openai
from bertopic.representation import OpenAI

prompt = """
I have a topic that contains the following documents:
[DOCUMENTS]

The topic is described by the following keywords: [KEYWORDS]

Based on the information above, extract a short topic label in the following
format:
topic: <short topic label>
"""
# Update our topic representations using GPT-3.5
client = openai.OpenAI(api_key="sk[redacted]4", base_url="https://api.deepseek.com")
representation_model = OpenAI(
    client, model="deepseek-chat", exponential_backoff=True, chat=True,
prompt=prompt
)
topic_model.update_topics(abstracts, representation_model=representation_model)

# Show topic differences
topic_differences(topic_model, original_topics)
```

```
100%|[redacted]| 155/155 [11:20<00:00, 4.39s/it]
```

| | Topic | Original | Updated |
|---|---|---|---|
| 0 | 0 | question\|qa\|answer\|questions\|answering | Question Answering and Reading Comprehension i... |
| 1 | 1 | speech\|asr\|recognition\|end\|acoustic | Speech Processing and End-to-End Models for Tr... |
| 2 | 2 | hate\|offensive\|speech\|detection\|toxic | Multilingual and Context-Aware Hate Speech Det... |
| 3 | 3 | summarization\|summaries\|summary\|abstractive\|do... | Neural Approaches to Document Summarization |
| 4 | 4 | gender\|bias\|biases\|debiasing\|fairness | Gender Bias in NLP and AI Language Models |

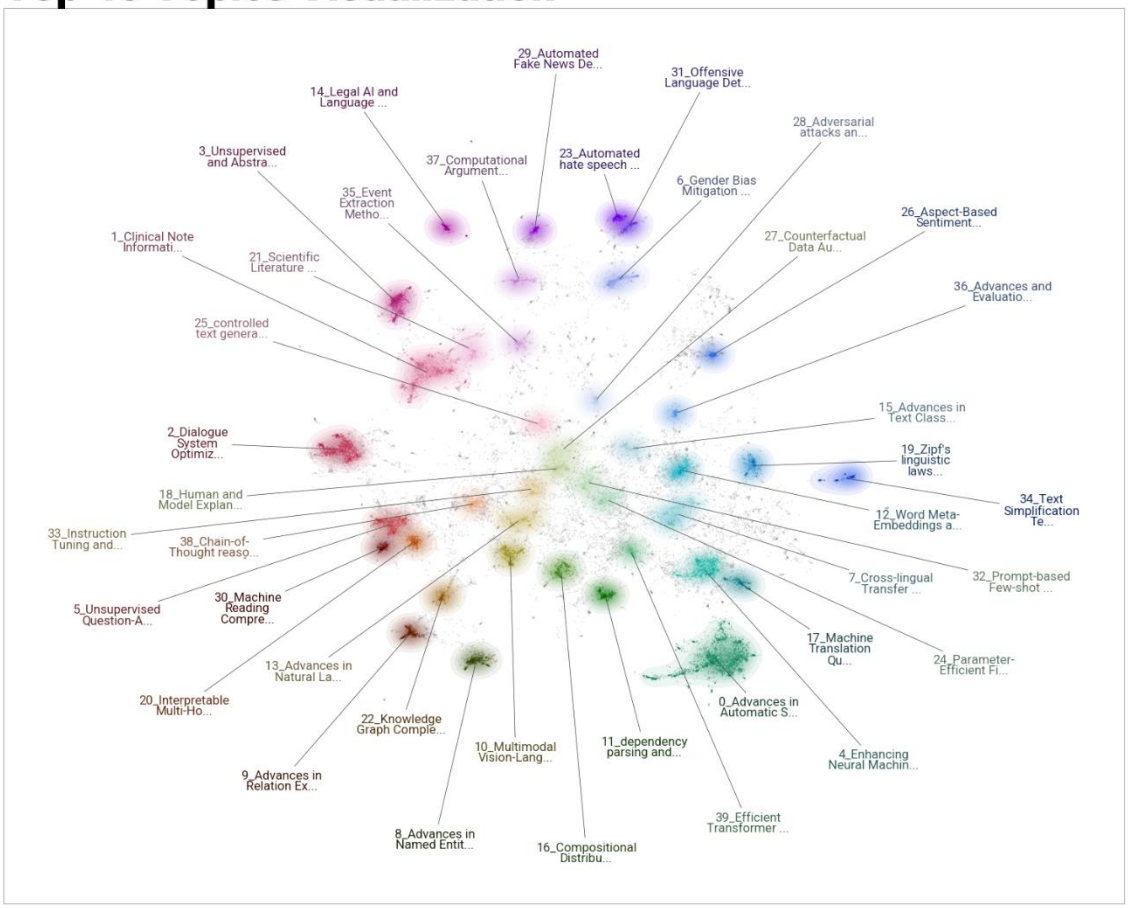
对每个主题：

1. 选取代表性文档（同 Flan-T5 逻辑）
2. 提取主题关键词
3. 填充提示模板
4. 发送到 DeepSeek 模型

5. 解析返回的标签格式 topic：生成标签

6. 用生成标签更新主题表示

```
import datamapplot
import numpy as np
import matplotlib

matplotlib.rcParams["figure.dpi"] = 300

fig, ax = datamapplot.create_plot(
        filtered_embeddings,
        formatted_labels,
        figsize=(15,12),
        title="Top 40 Topics Visualization",
        )
```



# Top 40 Topics Visualization